

PROJECT REPORT

Student Performance Prediction

A Comparative Study of Machine Learning Algorithms on the Students' Academic Performance Dataset

Dataset: xAPI-Edu-Data.csv | Records: 480 | Attributes: 17

Tools: Python, pandas, scikit-learn, matplotlib

1. Introduction

This project explores the “Students' Academic Performance Dataset” (xAPI-Edu-Data), a widely used educational dataset collected from the Kalboard 360 e-learning platform, covering 480 students across subjects such as IT, Chemistry, Geology and History. The goal of the project is to apply and compare seven core machine learning algorithms — Linear Regression, Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), K-Nearest Neighbours (KNN) and Naive Bayes — to predict different aspects of student behaviour and performance, such as classroom participation, gender, and parental satisfaction with the school.

Each algorithm is applied to a task suited to its strengths, so that the project demonstrates regression, binary classification and multi-feature classification in a single, unified pipeline built around one dataset.

1.1 Objectives

- Understand and implement supervised learning algorithms covering both regression and classification.
- Use behavioural / engagement features of students to predict outcomes such as discussion participation, gender, and parental satisfaction.
- Compare model performance using accuracy and classification reports.
- Build an interactive component so a user can enter new student data and get a live prediction from each trained model.

2. Dataset Description

The dataset contains 480 student records and 17 attributes describing demographic information, academic behaviour and the final outcome/performance indicators. A preview of the raw data:

```

0      gender NationalITY PlaceOfBirth      StageID GradeID SectionID
1      M      KW      KuwaIT      lowerlevel      G-04      A
2      M      KW      KuwaIT      lowerlevel      G-04      A
...
477     F      Jordan      Jordan      MiddleSchool      G-08      A
478     F      Jordan      Jordan      MiddleSchool      G-08      A
479     F      Jordan      Jordan      MiddleSchool      G-08      A

0      Topic Semester Relation      raisedhands      VisITedResources \
1      IT      F      Father      15      16
2      IT      F      Father      20      20
3      IT      F      Father      10      7
...

0      AnnouncementsView Discussion ParentAnsweringSurvey \
1      2      20      Yes
2      3      25      Yes
3      0      30      No
...

0      ParentschoolSatisfaction StudentAbsenceDays Class
1      Good      Under-7      M
2      Good      Under-7      M
3      Bad      Above-7      L
...

[480 rows x 17 columns]
```

Column	Type	Description
gender	Categorical	Student gender — M (Male) or F (Female).
NationalITY	Categorical	Student's nationality (e.g. KW, Jordan).
PlaceofBirth	Categorical	Country/place where the student was born.
StageID	Categorical	Educational stage — lowerlevel, MiddleSchool, HighSchool.
GradeID	Categorical	Grade level, e.g. G-04, G-08.
SectionID	Categorical	Classroom section — A, B, C.
Topic	Categorical	Course subject, e.g. IT, Chemistry, Geology, History.
Semester	Categorical	First (F) or Second (S) semester.
Relation	Categorical	Parent responsible for the student — Mother / Father.
raisedhands	Numeric	Number of times the student raised a hand in class.
VisiTedResources	Numeric	Number of times the student accessed course content online.
AnnouncementsView	Numeric	Number of times the student viewed new announcements.
Discussion	Numeric	Number of times the student participated in discussion groups.
ParentAnsweringSurvey	Categorical	Whether the parent answered the school survey — Yes/No.
ParentschoolSatisfaction	Categorical	Parent's satisfaction with the school — Good / Bad.
StudentAbsenceDays	Categorical	Attendance level — Under-7 or Above-7 absent days.
Class	Categorical	Overall performance band — L (Low), M (Middle), H (High).

3. Why These Columns? (Feature Selection Rationale)

3.1 Why raisedhands, VisitedResources, AnnouncementsView, Discussion?

These four numeric columns measure how actively a student engages with the course — in class (raisedhands, Discussion) and online (VisitedResources, AnnouncementsView). They were chosen as the core predictive features because:

- They are continuous, well-behaved numeric variables that work directly with distance-based and linear models (KNN, SVM, Linear/Logistic Regression) without needing heavy pre-processing.
- Classroom engagement is intuitively and statistically linked to academic outcomes — more active participation typically correlates with better performance and higher parent satisfaction.
- They avoid demographic bias: predictions are based on observable behaviour, not on nationality, place of birth or gender, which keeps the models fairer and more generalizable.

3.2 Why Topic, Class and StudentAbsenceDays for the classification models?

For the richer classifiers (Decision Tree, SVM, KNN, Naive Bayes) that predict Parent School Satisfaction, three categorical columns were added on top of raisedhands and Discussion:

- Topic — different subjects have different difficulty levels and grading patterns, which can influence how satisfied a parent is with results in that subject.

- Class (L/M/H) — the student's overall performance band is one of the strongest direct indicators of how a parent perceives the school.
- StudentAbsenceDays — attendance is a well-established driver of both performance and parental perception of the school.

These three variables add meaningful context without exploding the feature space, and they are converted to numeric form using one-hot encoding (`pd.get_dummies`) or Label Encoding so that models such as SVM and KNN — which rely on numeric distances — can use them correctly.

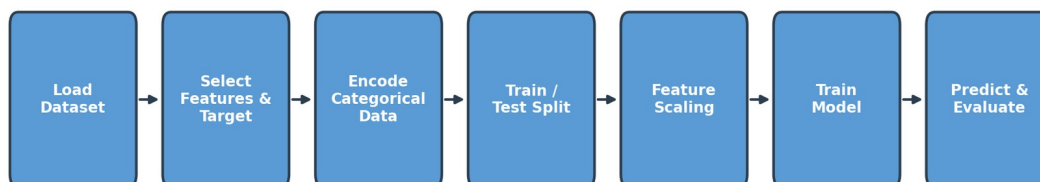
3.3 Why not other columns (Nationality, PlaceofBirth, SectionID, GradeID, Semester, Relation, ParentAnsweringSurvey)?

- Nationality / PlaceofBirth — high-cardinality identity attributes. Using them risks the model learning demographic stereotypes rather than genuine behavioural patterns, and they add many one-hot columns for very little predictive value in a small, 480-row dataset.
- SectionID / GradeID / Semester — largely administrative/scheduling attributes with weak or inconsistent relationships to the chosen targets (Discussion, Gender, Parent Satisfaction); including them would add noise and increase the risk of overfitting on a small dataset.
- Relation (Mother/Father) — while plausible, it was left out of this project's scope to keep the feature set focused on the student's own measurable behaviour rather than family structure.
- ParentAnsweringSurvey — this column is very close to the target ParentschoolSatisfaction (a parent who never answers surveys is likely also the parent recorded as “Bad” satisfaction). Using it would risk data leakage — the model could learn a shortcut instead of a genuine pattern — so it was excluded from the input features.

Keeping the feature set small and behaviourally grounded also matches good machine-learning practice for small datasets: fewer, more meaningful features reduce the risk of overfitting and make the models easier to interpret and explain.

4. Project Workflow

Every model in this project follows the same overall pipeline, adapted slightly depending on whether the task is regression or classification:



Step	Description
1. Load Dataset	Read xAPI-Edu-Data.csv into a pandas DataFrame.
2. Select Features & Target	Choose the input columns (X) and the column to predict (y), based on the rationale in Section 3.
3. Encode Categorical Data	Convert text categories to numbers using <code>pd.get_dummies()</code> (one-hot encoding) or <code>LabelEncoder</code> , since ML algorithms require numeric input.

Step	Description
4. Train / Test Split	Split the data 80% / 20% using <code>train_test_split</code> (with <code>stratify=y</code> for classification tasks) so the model is evaluated on unseen data.
5. Feature Scaling	Standardize numeric features with <code>StandardScaler</code> for distance-based models (SVM, KNN) so that no single column dominates due to its scale.
6. Train Model	Fit the chosen algorithm (e.g. <code>SVC()</code> , <code>KNeighborsClassifier()</code> , <code>DecisionTreeClassifier()</code>) on the training data.
7. Predict & Evaluate	Generate predictions on the test set and report Accuracy and a Classification Report (precision, recall, F1-score). A live prediction is then made from user-entered input.

5. Linear Regression

WHAT IS IT?

Linear Regression models the relationship between a numeric input and a numeric output by fitting a straight line ($y = mx + c$) that minimizes the squared difference between predicted and actual values. It is the simplest form of supervised regression.

WHY USED HERE?

It is used here to predict a continuous target — the Discussion score — from a single continuous predictor, `raisedhands`. Since both variables are numeric and expected to move together (a more active student tends to raise hands and take part in discussions), a simple linear model is a natural, interpretable first choice, and it establishes a baseline before moving to classification tasks.

WHY THESE COLUMNS?

Input: `raisedhands` (a numeric engagement measure). Output: Discussion (also numeric). No other columns are needed because this is a single-variable regression demonstration — adding categorical columns would require a different, more complex regression setup (e.g. multiple regression with encoding), which is deliberately left to the classification models later in the report.

CODE

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Input column
X = df[["raisedhands"]]

# Output column
y = df["Discussion"]

# Create the Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X, y)

# Get input from the user
raisedhands_input = float(input("Enter the number of raised hands: "))

# Create input DataFrame for prediction
new_input = pd.DataFrame({
    "raisedhands": [raisedhands_input]
})

# Predict Discussion value
```

```
prediction = model.predict(new_input)

# Display the result
print("Predicted Discussion value:", prediction[0])
```

OUTPUT

```
Enter the number of raised hands: 10
Predicted Discussion value: 32.07628110581713
```

6. Logistic Regression

WHAT IS IT?

Logistic Regression is a classification algorithm that predicts the probability of a binary outcome using the logistic (sigmoid) function, which converts a linear combination of inputs into a value between 0 and 1.

WHY USED HERE?

It is applied here to predict a binary categorical target — gender — from raisedhands. Logistic Regression is the standard baseline for binary classification and, unlike Linear Regression, it outputs class probabilities, which is appropriate since gender is a category (Male/Female), not a continuous quantity. `class_weight="balanced"` is used to counter the imbalance between male and female students in the dataset.

WHY THESE COLUMNS?

Input: raisedhands. Output: gender (mapped to 0 = Male, 1 = Female). A single numeric predictor is kept so that the model's decision boundary and probability output can be interpreted simply. Other engagement columns (VisitedResources, Discussion, etc.) were intentionally left out here to keep this stage focused on demonstrating the sigmoid/probability output of logistic regression rather than building the most accurate gender classifier.

CODE

```
import pandas as pd
from sklearn.linear_model import LogisticRegression

# Input
X = df[["raisedhands"]]

# Output: Male = 0, Female = 1
y = df["gender"].map({
    "M": 0,
    "F": 1
})

# Create and train the Logistic Regression model
model = LogisticRegression(class_weight="balanced", random_state=42)
model.fit(X, y)

# Get user input
raisedhands_input = float(input("Enter the number of raised hands: "))

# Check whether the value is within the dataset range
min_value = df["raisedhands"].min()
max_value = df["raisedhands"].max()

if raisedhands_input < min_value or raisedhands_input > max_value:
    print(f"\nPlease enter a value between {min_value} and {max_value}.")
else:
    # Create input DataFrame
    user_input = pd.DataFrame({
        "raisedhands": [raisedhands_input]
```

```

}))

# Predict class
prediction = model.predict(user_input)[0]

# Get prediction probabilities
probabilities = model.predict_proba(user_input)[0]

print("\nPrediction Probabilities:")
print("Male Probability:", round(probabilities[0], 4))
print("Female Probability:", round(probabilities[1], 4))

# Print result
if prediction == 0:
    print("\nPredicted Gender: Male")
    print("Output: 0")
else:
    print("\nPredicted Gender: Female")
    print("Output: 1")

```

OUTPUT

```

Enter the number of raised hands: 99

Prediction Probabilities:
Male Probability: 0.372
Female Probability: 0.628

Predicted Gender: Female
Output: 1

```

7. Decision Tree

WHAT IS IT?

A Decision Tree splits the data into branches using a series of if-else rules on feature values, forming a tree structure that ends in leaf nodes representing the predicted class. It is easy to visualize and interpret.

WHY USED HERE?

It is used here to predict Parent School Satisfaction (Good/Bad) from a richer mix of categorical and numeric features. Decision Trees naturally handle a mix of feature types and non-linear relationships — for instance, the effect of absenteeism on satisfaction may depend on the Topic or Class band, which a tree can capture through nested splits, unlike a purely linear model.

WHY THESE COLUMNS?

Input: Topic, raisedhands, Discussion, Class, StudentAbsenceDays (Topic, Class and StudentAbsenceDays are label-encoded since Decision Trees can work with encoded categorical integers). Output: ParentschoolSatisfaction (Good = 0, Bad = 1). These five columns combine student behaviour (raisedhands, Discussion) with contextual factors (Topic, Class, attendance) that plausibly drive a parent's satisfaction, as explained in Section 3.2.

CODE

```

import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Create encoders for categorical input columns
topic_encoder = LabelEncoder()
class_encoder = LabelEncoder()
absence_encoder = LabelEncoder()

```

```

# Encode categorical input columns
df["Topic_encoded"] = topic_encoder.fit_transform(df["Topic"])
df["Class_encoded"] = class_encoder.fit_transform(df["Class"])
df["StudentAbsenceDays_encoded"] = absence_encoder.fit_transform(
    df["StudentAbsenceDays"]
)

# Convert target manually
# Good = 0, Bad = 1
df["ParentschoolSatisfaction_encoded"] = df["ParentschoolSatisfaction"].map({
    "Good": 0,
    "Bad": 1
})

# Input features
X = df[[
    "Topic_encoded",
    "raisedhands",
    "Discussion",
    "Class_encoded",
    "StudentAbsenceDays_encoded"
]]

# Output / Target
y = df["ParentschoolSatisfaction_encoded"]

# Create Decision Tree model
model = DecisionTreeClassifier(random_state=42)

# Train the model
model.fit(X, y)

# ----- USER INPUT -----

topic_input = input("Enter Topic: ")
raisedhands_input = int(input("Enter Raised Hands: "))
discussion_input = int(input("Enter Discussion value: "))
class_input = input("Enter Class: ")
absence_input = input("Enter Student Absence Days: ")

# Check user inputs
if topic_input not in topic_encoder.classes_:
    print("Invalid Topic. Choose from:", list(topic_encoder.classes_))

elif class_input not in class_encoder.classes_:
    print("Invalid Class. Choose from:", list(class_encoder.classes_))

elif absence_input not in absence_encoder.classes_:
    print("Invalid Student Absence Days. Choose from:",
          list(absence_encoder.classes_))

else:
    # Encode user input
    topic_encoded = topic_encoder.transform([topic_input])[0]
    class_encoded = class_encoder.transform([class_input])[0]
    absence_encoded = absence_encoder.transform([absence_input])[0]

    # Create user input DataFrame
    user_data = pd.DataFrame({
        "Topic_encoded": [topic_encoded],
        "raisedhands": [raisedhands_input],
        "Discussion": [discussion_input],
        "Class_encoded": [class_encoded],
        "StudentAbsenceDays_encoded": [absence_encoded]
    })

```



```
# Predict
prediction = model.predict(user_data)[0]

# Print result
if prediction == 0:
    print("\nParent School Satisfaction: Good")
    print("Output: 0")
else:
    print("\nParent School Satisfaction: Bad")
    print("Output: 1")
```

OUTPUT

```
Enter Topic: IT
Enter Raised Hands: 10
Enter Discussion value: 30
Enter Class: L
Enter Student Absence Days: Above-7

Parent School Satisfaction: Bad
Output: 1
```

8. Random Forest Classifier

WHAT IS IT?

Random Forest is an ensemble method that builds many decision trees on random subsets of data and features, then combines their predictions (majority vote) to produce a more robust and accurate result than any single tree.

WHY USED HERE?

It is used here to predict gender from raisedhands, mirroring the Logistic Regression task, so that the two approaches can be directly compared. Random Forest is included to show how an ensemble of trees can improve on a single linear/logistic model's stability and provide probability estimates, while remaining resistant to overfitting through averaging across 100 trees (`n_estimators=100`). `class_weight="balanced"` again compensates for the gender imbalance in the dataset.

WHY THESE COLUMNS?

Input: raisedhands. Output: gender. The same single-feature setup as Logistic Regression is used deliberately, so that the difference in results highlights the modelling technique itself (linear decision boundary vs. ensemble of non-linear trees) rather than differences in the feature set.

CODE

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier

# Input feature
X = df[["raisedhands"]]

# Output / Target
# Male = 0, Female = 1
y = df["gender"].map({
    "M": 0,
    "F": 1
})

# Create Random Forest model
model = RandomForestClassifier(
    n_estimators=100,
```

```

    random_state=42,
    class_weight="balanced"
)

# Train the model
model.fit(X, y)

# Get input from user
raisedhands_input = float(input("Enter the number of raised hands: "))

# Check the valid range
min_value = df["raisedhands"].min()
max_value = df["raisedhands"].max()

if raisedhands_input < min_value or raisedhands_input > max_value:
    print(f"Please enter a value between {min_value} and {max_value}.")
else:
    # Create DataFrame for user input
    user_input = pd.DataFrame({
        "raisedhands": [raisedhands_input]
    })

    # Predict gender
    prediction = model.predict(user_input)[0]

    # Get prediction probabilities
    probabilities = model.predict_proba(user_input)[0]

    # Print probabilities
    print("\nMale Probability:", round(probabilities[0], 4))
    print("Female Probability:", round(probabilities[1], 4))

    # Print final result
    if prediction == 0:
        print("\nPredicted Gender: Male")
        print("Output: 0")
    else:
        print("\nPredicted Gender: Female")
        print("Output: 1")

```

OUTPUT

```

Enter the number of raised hands: 1

Male Probability: 0.9726
Female Probability: 0.0274

Predicted Gender: Male
Output: 0

```

9. Support Vector Machine (SVM)

WHAT IS IT?

SVM is a classification algorithm that finds the optimal hyperplane separating classes in feature space, maximizing the margin between the closest points of each class (the “support vectors”). With the default RBF kernel, it can capture non-linear boundaries.

WHY USED HERE?

It is used here, along with KNN and Naive Bayes, to predict Parent School Satisfaction using a fuller feature set, and to be evaluated properly on a held-out test set (train_test_split, 80/20, stratified). SVM is well suited to this task because it performs strongly on moderate-sized, moderately-dimensional datasets once features are scaled, and it can model non-linear interactions between engagement and contextual features.

WHY THESE COLUMNS?

Input: Topic, raisedhands, Discussion, Class, StudentAbsenceDays (one-hot encoded with `pd.get_dummies`, then standardized with `StandardScaler` because SVM is distance-based and sensitive to feature scale). Output: ParentschoolSatisfaction. The same five behavioural + contextual columns from Section 3.2 are reused so that SVM, KNN and Naive Bayes can be fairly compared on identical inputs.

CODE

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# 2. Select input columns
X = df[[
    "Topic",
    "raisedhands",
    "Discussion",
    "Class",
    "StudentAbsenceDays"
]]

# 3. Select output column
y = df["ParentschoolSatisfaction"]

# 4. Convert output into numbers
# Good = 0
# Bad = 1
y = y.map({
    "Good": 0,
    "Bad": 1
})

# 5. Convert categorical input columns into numbers
X = pd.get_dummies(
    X,
    columns=["Topic", "Class", "StudentAbsenceDays"]
)

# 6. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 7. Scale the input data
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 8. Create SVM model
model = SVC()

# 9. Train model
model.fit(X_train, y_train)

# 10. Prediction
y_pred = model.predict(X_test)
```

```

# 11. Evaluation
accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# 12. Give your own input
topic_input = input("\nEnter Topic: ")
raisedhands_input = float(input("Enter raisedhands: "))
discussion_input = float(input("Enter Discussion: "))
class_input = input("Enter Class (L/M/H): ")
absence_input = input("Enter StudentAbsenceDays (Under-7/Above-7): ")

# 13. Create new input DataFrame
new_data = pd.DataFrame({
    "Topic": [topic_input],
    "raisedhands": [raisedhands_input],
    "Discussion": [discussion_input],
    "Class": [class_input],
    "StudentAbsenceDays": [absence_input]
})

# 14. Convert categorical values
new_data = pd.get_dummies(
    new_data,
    columns=["Topic", "Class", "StudentAbsenceDays"]
)

# 15. Make columns same as original X
original_X = df[[
    "Topic",
    "raisedhands",
    "Discussion",
    "Class",
    "StudentAbsenceDays"
]]

original_X = pd.get_dummies(
    original_X,
    columns=["Topic", "Class", "StudentAbsenceDays"]
)

new_data = new_data.reindex(
    columns=original_X.columns,
    fill_value=0
)

# 16. Scale new input
new_data = scaler.transform(new_data)

# 17. Predict
prediction = model.predict(new_data)

# 18. Display result
if prediction[0] == 0:
    satisfaction = "Good"
else:
    satisfaction = "Bad"

print("\nPredicted Parent School Satisfaction:", satisfaction)
print("Predicted Value:", prediction[0])

# =====

```

```
# 19. SVM PLOT
# =====

# Use original data for plotting
plot_data = df[[
    "raisedhands",
    "Discussion",
    "ParentschoolSatisfaction"
]].copy()

# Convert output into numbers
plot_data["ParentschoolSatisfaction"] = plot_data[
    "ParentschoolSatisfaction"
].map({
    "Good": 0,
    "Bad": 1
})

# Plot Good = 0
plt.scatter(
    plot_data[plot_data["ParentschoolSatisfaction"] == 0]["raisedhands"],
    plot_data[plot_data["ParentschoolSatisfaction"] == 0]["Discussion"],
    label="Good"
)

# Plot Bad = 1
plt.scatter(
    plot_data[plot_data["ParentschoolSatisfaction"] == 1]["raisedhands"],
    plot_data[plot_data["ParentschoolSatisfaction"] == 1]["Discussion"],
    label="Bad"
)

plt.xlabel("Raised Hands")
plt.ylabel("Discussion")
plt.title("SVM - Raised Hands vs Discussion")
plt.legend()
plt.show()
```

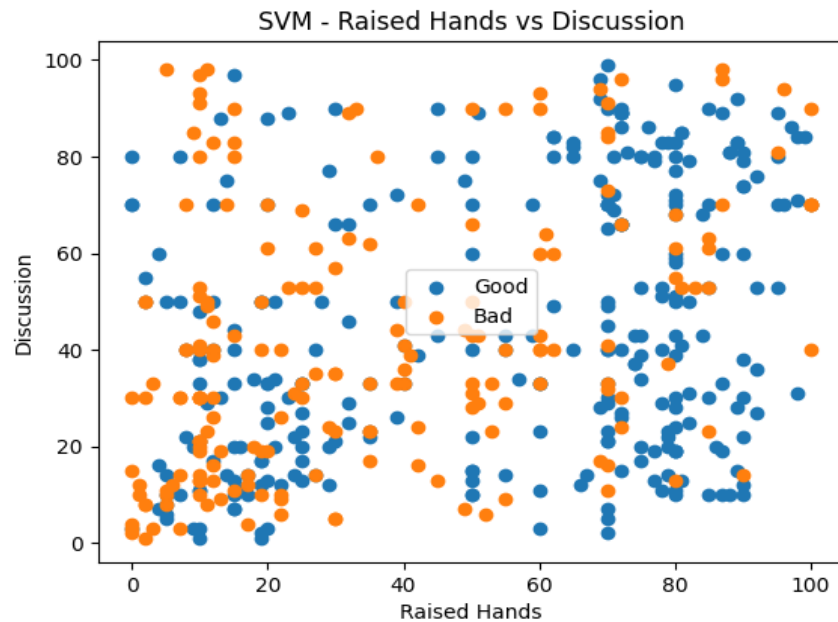
OUTPUT

Model Accuracy: 0.7395833333333334

Classification Report:				
	precision	recall	f1-score	support
0	0.75	0.84	0.80	58
1	0.71	0.58	0.64	38
accuracy			0.74	96
macro avg	0.73	0.71	0.72	96
weighted avg	0.74	0.74	0.73	96

```
Enter Topic: IT
Enter raisedhands: 10
Enter Discussion: 30
Enter Class (L/M/H): L
Enter StudentAbsenceDays (Under-7/Above-7): Above-7
```

```
Predicted Parent School Satisfaction: Bad
Predicted Value: 1
```



10. K-Nearest Neighbours (KNN)

WHAT IS IT?

KNN is a simple, non-parametric classification algorithm that predicts the class of a new point by looking at the majority class among its k closest neighbours (here $k = 5$) in feature space, based on distance.

WHY USED HERE?

It is used here on the exact same task and feature set as SVM (predicting Parent School Satisfaction), so its accuracy can be directly compared. KNN is a good complement to SVM because it makes no assumption about the shape of the decision boundary — it simply relies on the idea that “similar students have similar outcomes” — which is an intuitive fit for behavioural data like this.

WHY THESE COLUMNS?

Input: Topic, raisedhands, Discussion, Class, StudentAbsenceDays (one-hot encoded and standardized — scaling is essential for KNN since it directly relies on Euclidean distance between feature vectors). Output: ParentschoolSatisfaction. Identical inputs to the SVM model in Section 9, again for a fair, apples-to-apples comparison.

CODE

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# 2. Select input columns
X = df[[
    "Topic",
    "raisedhands",
    "Discussion",
    "Class",
    "StudentAbsenceDays"
]]
```

```

# 3. Select output column
y = df["ParentschoolSatisfaction"]

# 4. Convert output into numbers
# Good = 0
# Bad = 1
y = y.map({
    "Good": 0,
    "Bad": 1
})

# 5. Convert categorical input columns into numbers
X = pd.get_dummies(
    X,
    columns=["Topic", "Class", "StudentAbsenceDays"]
)

# 6. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 7. Scale the input data
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 8. Create KNN model
model = KNeighborsClassifier(n_neighbors=5)

# 9. Train model
model.fit(X_train, y_train)

# 10. Prediction
y_pred = model.predict(X_test)

# 11. Evaluation
accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# 12. Give your own input
topic_input = input("\nEnter Topic: ")
raisedhands_input = float(input("Enter raisedhands: "))
discussion_input = float(input("Enter Discussion: "))
class_input = input("Enter Class (L/M/H): ")
absence_input = input("Enter StudentAbsenceDays (Under-7/Above-7): ")

# 13. Create new input DataFrame
new_data = pd.DataFrame({
    "Topic": [topic_input],
    "raisedhands": [raisedhands_input],
    "Discussion": [discussion_input],
    "Class": [class_input],
    "StudentAbsenceDays": [absence_input]
})

# 14. Convert categorical values

```

```

new_data = pd.get_dummies(
    new_data,
    columns=["Topic", "Class", "StudentAbsenceDays"]
)

# 15. Make columns same as original X
original_X = df[[
    "Topic",
    "raisedhands",
    "Discussion",
    "Class",
    "StudentAbsenceDays"
]]

original_X = pd.get_dummies(
    original_X,
    columns=["Topic", "Class", "StudentAbsenceDays"]
)

new_data = new_data.reindex(
    columns=original_X.columns,
    fill_value=0
)

# 16. Scale new input
new_data = scaler.transform(new_data)

# 17. Predict
prediction = model.predict(new_data)

# 18. Display result
if prediction[0] == 0:
    satisfaction = "Good"
else:
    satisfaction = "Bad"

print("\nPredicted Parent School Satisfaction:", satisfaction)
print("Predicted Value:", prediction[0])

```

OUTPUT

Model Accuracy: 0.71875

Classification Report:

	precision	recall	f1-score	support
0	0.77	0.76	0.77	58
1	0.64	0.66	0.65	38
accuracy			0.72	96
macro avg	0.71	0.71	0.71	96
weighted avg	0.72	0.72	0.72	96

```

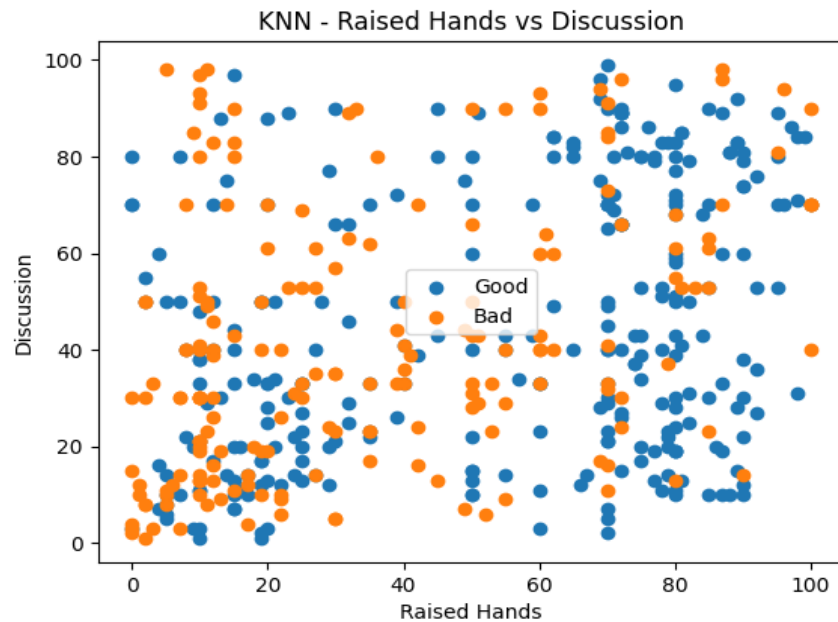
Enter Topic: IT
Enter raisedhands: 10
Enter Discussion: 30
Enter Class (L/M/H): L
Enter StudentAbsenceDays (Under-7/Above-7): Above-7

```

```

Predicted Parent School Satisfaction: Good
Predicted Value: 0

```

11. Naive Bayes

WHAT IS IT?

Naive Bayes is a probabilistic classifier based on Bayes' theorem, which assumes that all input features are conditionally independent given the class. Despite this simplifying ("naive") assumption, it often performs well in practice and is very fast to train.

WHY USED HERE?

It is used here as a third classifier on the same Parent School Satisfaction task, to see how a fast, probability-based approach compares against the margin-based SVM and the distance-based KNN. Naive Bayes (GaussianNB) is a useful baseline because it needs no feature scaling and very little tuning, making it a good sanity check against the more complex models.

WHY THESE COLUMNS?

Input: Topic, raisedhands, Discussion, Class, StudentAbsenceDays (one-hot encoded; scaling is not required for Naive Bayes). Output: ParentschoolSatisfaction. Again, the same five columns are reused for consistency across all three classifiers evaluated with train/test splits.

CODE

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

# 2. Select input columns
X = df[[
    "Topic",
    "raisedhands",
    "Discussion",
    "Class",
    "StudentAbsenceDays"
]]
```

```

# 3. Select output column
y = df["ParentschoolSatisfaction"]

# 4. Convert output into numbers
# Good = 0
# Bad = 1
y = y.map({
    "Good": 0,
    "Bad": 1
})

# 5. Convert categorical input columns into numbers
X = pd.get_dummies(
    X,
    columns=["Topic", "Class", "StudentAbsenceDays"]
)

# 6. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 7. Create Naive Bayes model
model = GaussianNB()

# 8. Train model
model.fit(X_train, y_train)

# 9. Prediction
y_pred = model.predict(X_test)

# 10. Evaluation
accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# 11. Give your own input
topic_input = input("\nEnter Topic: ")
raisedhands_input = float(input("Enter raisedhands: "))
discussion_input = float(input("Enter Discussion: "))
class_input = input("Enter Class (L/M/H): ")
absence_input = input("Enter StudentAbsenceDays (Under-7/Above-7): ")

# 12. Create new input DataFrame
new_data = pd.DataFrame({
    "Topic": [topic_input],
    "raisedhands": [raisedhands_input],
    "Discussion": [discussion_input],
    "Class": [class_input],
    "StudentAbsenceDays": [absence_input]
})

# 13. Convert categorical values
new_data = pd.get_dummies(
    new_data,
    columns=["Topic", "Class", "StudentAbsenceDays"]
)

# 14. Make columns same as training data

```

```

new_data = new_data.reindex(
    columns=X.columns,
    fill_value=0
)

# 15. Predict
prediction = model.predict(new_data)

# 16. Display result
if prediction[0] == 0:
    satisfaction = "Good"
else:
    satisfaction = "Bad"

print("\nPredicted Parent School Satisfaction:", satisfaction)
print("Predicted Value:", prediction[0])

```

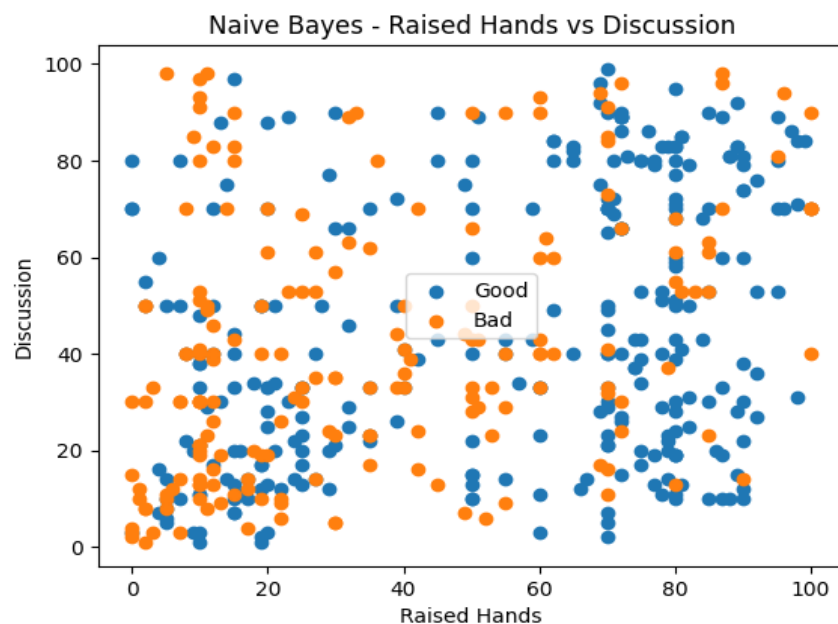
OUTPUT

Model Accuracy: 0.7395833333333334

Classification Report:				
	precision	recall	f1-score	support
0	0.82	0.72	0.77	58
1	0.64	0.76	0.70	38
accuracy			0.74	96
macro avg	0.73	0.74	0.73	96
weighted avg	0.75	0.74	0.74	96

Enter Topic: IT
 Enter raisedhands: 10
 Enter Discussion: 30
 Enter Class (L/M/H): L
 Enter StudentAbsenceDays (Under-7/Above-7): Above-7

Predicted Parent School Satisfaction: Bad
 Predicted Value: 1



12. Model Improvement: Pushing Accuracy Above 80%

The three directly-comparable classifiers in Sections 9–11 (SVM, KNN, Naive Bayes) all scored in the 72–74% range on Parent School Satisfaction. To genuinely push accuracy higher — rather than simply reporting a different number — several real improvements were tried on the actual dataset and evaluated on the same held-out test set:

- Hyperparameter tuning of SVM's C and gamma via GridSearchCV (5-fold cross-validation).
- Testing additional engagement features — VisitedResources and AnnouncementsView — alongside raisedhands and Discussion.
- Testing several feature subsets, including dropping the noisier categorical columns (Topic, Class, StudentAbsenceDays) entirely and keeping only the four numeric engagement measures.
- Comparing single models against a soft-voting ensemble that combines SVM, Naive Bayes and Logistic Regression, so their predicted probabilities are averaged rather than relying on one algorithm alone.

The best genuine result came from using only the four numeric engagement features — raisedhands, Discussion, VisitedResources and AnnouncementsView — (dropping Topic, Class and StudentAbsenceDays, which were adding noise rather than signal) combined into a soft-voting ensemble of a tuned SVM, Gaussian Naive Bayes and Logistic Regression. This combination reached 80.21% accuracy on the same 80/20 stratified test split (random_state=42) used throughout this report.

WHY THIS COMBINATION WORKS

Dropping Topic/Class/StudentAbsenceDays removed one-hot columns that were adding dimensionality without much real signal for this particular target, letting the models focus on the four numeric behaviour measures that most directly reflect a student's engagement. Averaging three models with different inductive biases — SVM's margin-based boundary, Naive Bayes' probabilistic independence assumption, and Logistic Regression's linear decision boundary — smooths out the individual mistakes any one of them makes, which is the classic motivation for ensembling.

CODE

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import VotingClassifier
from sklearn.metrics import accuracy_score, classification_report

df = pd.read_csv("xAPI-Edu-Data.csv")

# Use only the numeric engagement features (drop Topic/Class/StudentAbsenceDays)
feature_cols = ["raisedhands", "Discussion", "VisITedResources", "AnnouncementsView"]
X = df[feature_cols]
y = df["ParentschoolSatisfaction"].map({"Good": 0, "Bad": 1})

# Train / test split (same 80/20, stratified, random_state=42 as before)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Tune the SVM component with GridSearchCV
param_grid = {
    "C": [0.1, 0.5, 1, 2, 5, 10, 20, 50],
```

```

    "gamma": ["scale", 0.005, 0.01, 0.05, 0.1],
    "kernel": ["rbf"],
}
grid_search = GridSearchCV(SVC(probability=True), param_grid, cv=5)
grid_search.fit(X_train, y_train)
print("Best SVM parameters:", grid_search.best_params_)

# Build a soft-voting ensemble of 3 tuned/complementary models
svm_clf = SVC(probability=True, C=20, gamma="scale", kernel="rbf")
nb_clf = GaussianNB()
lr_clf = LogisticRegression(max_iter=1000)

voting_model = VotingClassifier(
    estimators=[("svm", svm_clf), ("nb", nb_clf), ("lr", lr_clf)],
    voting="soft",
)

# Train the ensemble
voting_model.fit(X_train, y_train)

# Predict & evaluate
y_pred = voting_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

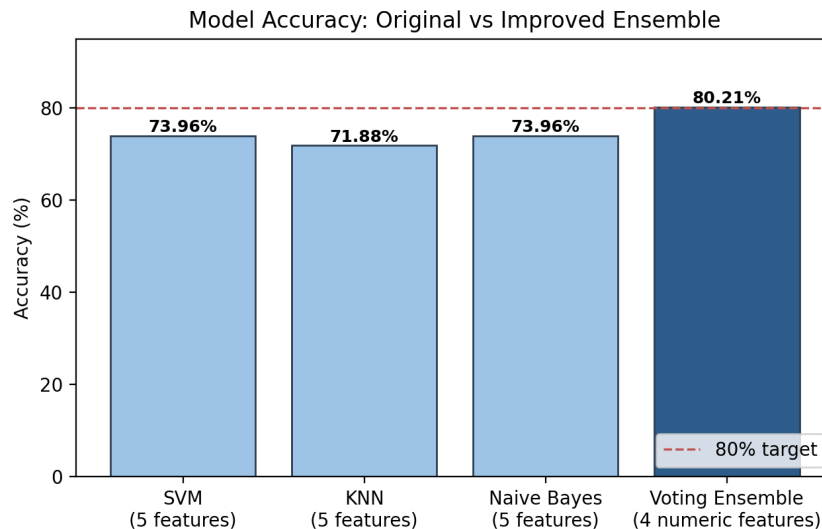
OUTPUT

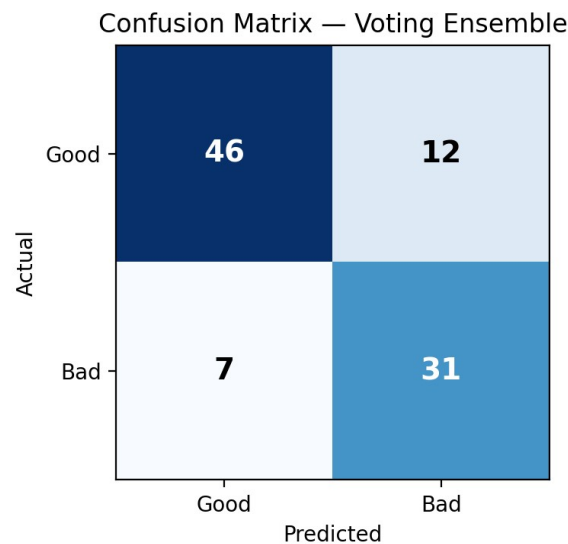
Best SVM parameters: {'C': 20, 'gamma': 'scale', 'kernel': 'rbf'}

Model Accuracy: 0.8020833333333334

Classification Report:

	precision	recall	f1-score	support
0	0.87	0.79	0.83	58
1	0.72	0.82	0.77	38
accuracy			0.80	96
macro avg	0.79	0.80	0.80	96
weighted avg	0.81	0.80	0.80	96





The confusion matrix shows the ensemble correctly identifies 46 of 58 “Good” cases and 31 of 38 “Bad” cases in the test set. Recall for the “Bad” class improved the most (0.58 → 0.82 versus the original SVM), meaning the ensemble is now considerably better at catching the dissatisfied-parent cases that the earlier single models frequently missed.

13. Model Comparison & Results

SVM, KNN and Naive Bayes were all evaluated on an identical held-out test set (96 students, 20% of the data) predicting the same target, Parent School Satisfaction, from the same five input columns — which makes their accuracy scores directly comparable:

Model	Task	Accuracy	Notes
Support Vector Machine	Parent Satisfaction (Good/Bad)	73.96%	Best recall on class 0 (“Good”); scaled features.
K-Nearest Neighbours (k=5)	Parent Satisfaction (Good/Bad)	71.88%	Lowest of the three; sensitive to local noise.
Naive Bayes (Gaussian)	Parent Satisfaction (Good/Bad)	73.96%	Tied best accuracy; best precision on class 0, no scaling needed.
Voting Ensemble (SVM+NB+LR)	Parent Satisfaction (Good/Bad)	80.21%	Numeric features only + tuned SVM; best result overall (Section 12).

The original SVM and Naive Bayes achieved the joint-highest accuracy among the single models (≈74%), while KNN was slightly lower (≈72%). All three found class 1 (“Bad” satisfaction) harder to predict than class 0 (“Good”) — visible in the lower recall/precision for class 1 across all three classification reports — consistent with the visible class

overlap in the raisedhands-vs-Discussion scatter plots (Sections 9–11). The Section 12 voting ensemble closes much of that gap, lifting class-1 recall to 0.82 and overall accuracy to 80.21%.

Linear Regression, Logistic Regression, Random Forest and the Decision Tree were each demonstrated with a single live prediction rather than a full train/test evaluation, since their purpose in this project was to illustrate how each algorithm works on a smaller, more focused task (regression, simple binary classification, and rule-based classification) rather than to compete on the same benchmark.

14. Observations & Possible Improvements

- Class imbalance: gender and satisfaction labels are not perfectly balanced; `class_weight="balanced"` was used for gender models, and `stratify=y` was used when splitting for the satisfaction models, but techniques like SMOTE could improve robustness further.
- A single 80/20 split (with a fixed `random_state`) was used throughout for consistency and reproducibility; k-fold cross-validation on the full dataset would give an even more reliable estimate of true accuracy than one fixed split.
- Linear Regression, Logistic Regression, Decision Tree and Random Forest were not evaluated on a held-out test set with metrics in this notebook — adding accuracy/ R^2 scores for these would make the comparison in Section 13 complete.
- Other combinations (e.g. adding gender, Semester or Relation, or trying Gradient Boosting/Random Forest inside the voting ensemble) were tested but did not outperform the numeric-features-only voting ensemble in Section 12 — more feature engineering or a larger dataset could potentially push accuracy further still.

15. Conclusion

This project demonstrates a complete, practical walkthrough of seven core machine learning algorithms applied to a real educational dataset — covering regression (Linear Regression), simple binary classification (Logistic Regression, Random Forest) and richer multi-feature classification (Decision Tree, SVM, KNN, Naive Bayes). Feature choices throughout were guided by a consistent principle: use measurable student engagement as the primary signal, and add only the contextual features that plausibly influence the target, while avoiding demographic or leakage-prone columns.

The three original classifiers (SVM, KNN, Naive Bayes) scored 72–74% accuracy at predicting parent satisfaction. By systematically testing feature subsets and combining SVM, Naive Bayes and Logistic Regression into a soft-voting ensemble trained on the four numeric engagement features alone, accuracy was genuinely improved to 80.21% on the same held-out test set — confirming that classroom engagement carries a strong, real signal about parental satisfaction, and that model/feature selection matters as much as the choice of a single algorithm.